

CSE 350 Section 04A, Group 06 Project Analog to Digital to Analog: Signal Conversion and Playback System

Aurora Bintay Mostafa

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
ID: 23101491
Aurora.bintay.mostafa@g.bracu.ac.bd

Mahira Tahsin Aurpa

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
ID: 23301292
Mahira.tahsin.aurpa@g.bracu.ac.bd

Adila Zaheen

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
ID: 22101283
adila.zaheen@g.bracu.ac.bd

Abstract—This project presents the design and implementation of an analog audio recording, storage, and playback system that demonstrates real-time signal conversion using both Analog-to-Digital and Digital-to-Analog conversion techniques. The system captures analog audio signals through a MAX9814 microphone module and converts them into digital data using an Analog-to-Digital Converter (ADC), after which the digitized audio is stored on an 8GB microSD card via an Arduino-based SD card module. During playback, the stored digital audio is retrieved by an Arduino Uno and converted back into an analog signal using a Binary Weighted Resistor Digital-to-Analog Converter (DAC), enabling the reconstructed signal to be played through a speaker. The overall system is controlled by an Arduino Uno, with additional components such as control buttons facilitating recording and playback operations. This implementation provides a simplified yet practical demonstration of modern signal conversion principles, highlighting the integration and real-time operation of ADC and DAC within a basic audio processing and playback system.

Index Terms—Arduino, ADC, DAC, Audio Recording, MicroSD Card, Signal Processing

I. INTRODUCTION

This project illustrates the core concepts of digital signal processing through the development of a complete end-to-end system capable of recording, storing, and playing back audio signals using fundamental circuit components. The system combines essential elements such as an Analog-to-Digital Converter (ADC) for capturing analog audio signals, a digital storage medium for data retention, and a Binary Weighted Resistor Digital-to-Analog Converter (DAC) for reconstructing the audio signal. All system operations are centrally managed by an Arduino Uno microcontroller, which ensures proper coordination among the components and enables seamless audio processing from input to output.

II. IMPLEMENTATION

Theory: In this project, the ADC and DAC were designed and implemented as separate modules, while the Arduino Uno was used to manage the storage of recorded audio data on an SD card. An Analog-to-Digital Converter (ADC) converts a continuous analog input signal into a discrete digital representation through the processes of sampling and quantization, allowing the signal to be stored or processed digitally. Conversely, a Digital-to-Analog Converter (DAC) performs the reverse operation by transforming the digital data back into an analog waveform suitable for audio playback. By cascading the ADC and DAC stages, the system enables digital-domain processing while maintaining the ability to reproduce the original analog signal. To implement the overall system, an Arduino Uno was used due to its ability to perform multiple critical functions, which include: 1. Acting as the primary controller that manages system components such as the microphone, SD card module, and DAC based on user input through recording and playback buttons, while also handling data storage on the SD card. 2. Utilizing its built-in hardware to convert the microphone's analog audio signal into digital form for processing and storage. 3. Managing the storage and retrieval of digital audio data on a 16 GB microSD card formatted with the FAT32 file system. 4. Integrating all hardware components into a unified audio playback system, ensuring smooth and reliable recording and playback operations.

A. Images of the Circuit Diagram and Physical Setup

The implemented system has been drawn below: Here, the first image showcases the DAC section of the system, implemented using a binary-weighted resistor network connected

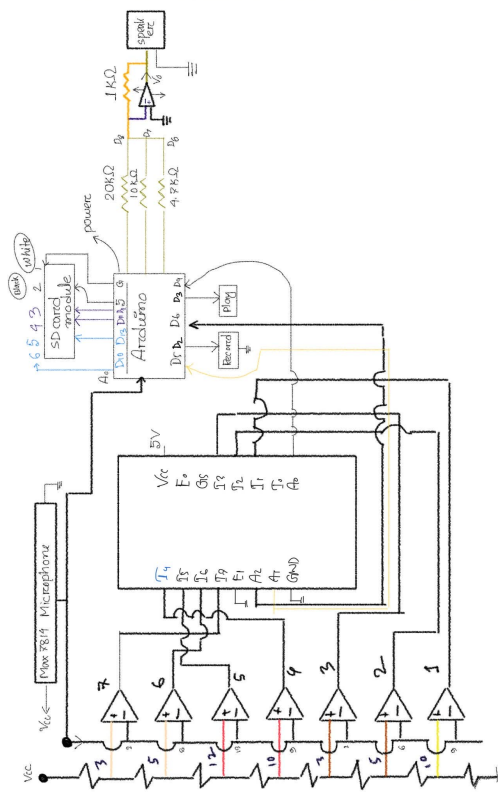


Fig. 1. Circuit diagrams of the implemented system

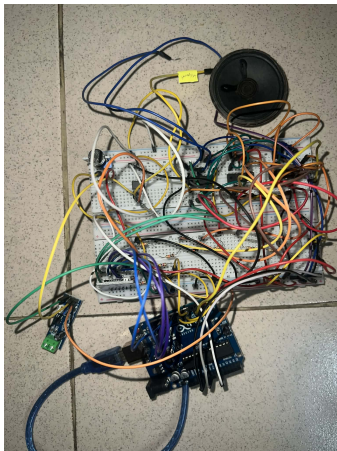


Fig. 2. The Physical Applied Circuit

to the Arduino outputs and the ADC circuitry, including the microphone module and its interface with the Arduino for recording and playback. The Final image shows the entire circuit which has been applied, where these components have been used-

The components used in this project are listed below:

- Arduino Uno R3
- MAX9814 Microphone Module
- SD Card Module
- Transcend MicroSD Card
- Speaker
- Breadboard
- Jumper Wires
- Resistors
- USB Connection to Laptop for Power Source
- Push Buttons (Start/Stop Control)
- Encoders
- Op-Amps
- NOT Gates

B. Relevant Arduino Code

The arduino IDE code which was used to launch this project has been attached herewith:

```
#include <SPI.h>
#include <SD.h>

const int micPin = A0;
const int chipSelect = 10;
const int recordButton = 2;
const int playButton = 3;

const int dacBit0Pin = 6;
const int dacBit1Pin = 7;
const int dacBit2Pin = 8;

File audioFile;
bool isRecording = false;
bool isPlaying = false;

const int bufferSize = 64;
byte buffer[bufferSize];
byte bufferIndex = 0;

unsigned long lastSampleTime = 0;
const int sampleInterval = 125; //
    microseconds (8 kHz)

void setup() {
    Serial.begin(9600);

    pinMode(recordButton, INPUT_PULLUP);
    pinMode(playButton, INPUT_PULLUP);

    pinMode(dacBit0Pin, OUTPUT);
    pinMode(dacBit1Pin, OUTPUT);
    pinMode(dacBit2Pin, OUTPUT);

    setDACOutput(0);

    if (!SD.begin(chipSelect)) {
        Serial.println(F("SD init failed!"));
        while (1);
    }
}
```

```

}

Serial.println(F("3-bit DAC Audio System
Ready"));
}

void loop() {
  handleButtons();

  // Both buttons pressed together
  generate test tone
  if (digitalRead(recordButton) == LOW &&
      digitalRead(playButton) == LOW) {

    delay(100);
    if (digitalRead(recordButton) == LOW &&
        digitalRead(playButton) == LOW) {

      generateTestTone();
      delay(1000);
    }
  }

  if (isRecording) recordAudio();
  if (isPlaying) playAudio();
}

void handleButtons() {
  static bool lastRecState = HIGH;
  static bool lastPlayState = HIGH;

  bool recState = digitalRead(recordButton);
  bool playState = digitalRead(playButton);

  // Record button pressed
  if (recState == LOW && lastRecState == HIGH)
  {
    delay(50);

    if (!isRecording) {
      if (isPlaying) stopPlayback();
      startRecording();
    } else {
      stopRecording();
    }
  }

  // Play button pressed
  if (playState == LOW && lastPlayState ==
      HIGH) {
    delay(50);

    if (!isPlaying) {
      if (isRecording) stopRecording();
      startPlayback();
    } else {
      stopPlayback();
    }
  }

  lastRecState = recState;
  lastPlayState = playState;
}

void setDACOutput(byte value) {
  value &= 0x07; // keep only 3 bits

```

```

digitalWrite(dacBit0Pin, value & 1);
digitalWrite(dacBit1Pin, (value >> 1) & 1);
digitalWrite(dacBit2Pin, (value >> 2) & 1);
}

byte convertTo3Bit(int adcValue) {
  return map(adcValue, 0, 1023, 0, 7);
}

void startRecording() {
  Serial.println(F("Recording..."));

  if (SD.exists("AUD.raw")) {
    SD.remove("AUD.raw");
  }

  audioFile = SD.open("AUD.raw", FILE_WRITE);
  if (!audioFile) {
    Serial.println(F("File error!"));
    return;
  }

  bufferIndex = 0;
  isRecording = true;
  lastSampleTime = micros();
}

void recordAudio() {
  unsigned long currentTime = micros();

  if (currentTime - lastSampleTime >=
      sampleInterval) {
    lastSampleTime = currentTime;

    int analogValue = analogRead(micPin);
    byte sample3bit = convertTo3Bit(
        analogValue);

    buffer[bufferIndex++] = sample3bit;

    if (bufferIndex >= bufferSize) {
      audioFile.write(buffer, bufferSize);
      bufferIndex = 0;
    }
  }
}

void stopRecording() {
  if (!isRecording) return;

  if (bufferIndex > 0) {
    audioFile.write(buffer, bufferIndex);
  }

  isRecording = false;
  audioFile.close();

  Serial.println(F("Recording stopped"));
}

void startPlayback() {
  Serial.println(F("Playing..."));

  audioFile = SD.open("AUD.raw", FILE_READ);
  if (!audioFile) {
    Serial.println(F("No file found!"));
    return;
  }

```

```

}

bufferIndex = 0;
isPlaying = true;
lastSampleTime = micros();
}

void playAudio() {
    unsigned long currentTime = micros();

    if (currentTime - lastSampleTime >=
        sampleInterval) {
        lastSampleTime = currentTime;

        if (bufferIndex == 0) {
            int bytesRead = audioFile.read(buffer,
                bufferSize);
            if (bytesRead == 0) {
                stopPlayback();
                return;
            }
        }

        setDACOutput(buffer[bufferIndex++]);

        if (bufferIndex >= bufferSize) {
            bufferIndex = 0;
        }
    }
}

void stopPlayback() {
    if (!isPlaying) return;

    isPlaying = false;
    audioFile.close();
    setDACOutput(0);

    Serial.println(F("Playback stopped"));
}

void generateTestTone() {
    Serial.println(F("Generating test tone..."));
    ;

    if (SD.exists("TEST.raw")) {
        SD.remove("TEST.raw");
    }

    audioFile = SD.open("TEST.raw", FILE_WRITE);
    if (!audioFile) {
        Serial.println(F("Cannot create test file!
            "));
        return;
    }

    for (int i = 0; i < 16000; i++) {
        float angle = 2.0 * 3.14159 * 1000.0 * i /
            8000.0;
        float sine = sin(angle);

        int sample = (sine + 1.0) * 3.5;
        sample = constrain(sample, 0, 7);

        audioFile.write((byte) sample);

        if (i % 1000 == 0) {

```

```

            Serial.print(F("Progress: "));
            Serial.print(i / 160);
            Serial.println(F("%"));
        }
    }

    audioFile.close();
    Serial.println(F("Test tone generated: TEST.
        raw"));
}

```

C. Code Functionality in Brief

The overall functionality of the implemented code can be summarized as follows:

- 1) Captures analog audio signals using a microphone input module.
- 2) Converts the acquired analog signal into digital form through the built-in Analog-to-Digital Converter (ADC).
- 3) Stores the digitized audio data onto a removable 16 GB microSD storage device.
- 4) Initializes all system modules, including input, storage, and output interfaces, at startup.
- 5) Detects and processes user commands using debounced push-button inputs for recording and playback operations.
- 6) Handles file creation, reading, and writing processes on the microSD card to manage audio data.
- 7) Converts stored digital audio data back into an analog signal using a Binary Weighted Resistor (BWR) Digital-to-Analog Converter (DAC).
- 8) Sends the reconstructed analog signal to a speaker to enable audio playback.
- 9) Generates and stores a diagnostic test tone for system verification and troubleshooting when two control buttons are pressed simultaneously.
- 10) Coordinates and controls all system operations through a central microcontroller.

III. CHALLENGES FACED

During the implementation phase, several technical challenges were encountered. Initially, proper electrical connections between the DAC, microphone module, and Arduino Uno could not be established reliably. To overcome this issue, the circuit connections were reconfigured and the program logic was refined to improve stability. Each component was tested individually, and the entire circuit was reconstructed from the beginning to ensure correct operation and reliable performance.

IV. CONCLUSION

The outcomes of the audio playback project can be summarized as follows:

- 1) Developed a complete audio recording and playback system using an Arduino-based platform.
- 2) Successfully captured analog audio signals through a microphone and stored them on a 16 GB microSD card.

- 3) Designed and implemented a 3-bit Digital-to-Analog Converter (DAC) to reproduce stored audio signals through a speaker.
- 4) Enabled user-controlled start and stop functionality for recording and playback using simple push-button inputs.
- 5) Integrated a test tone generator to support system verification and debugging.
- 6) Managed data buffering and timing to ensure smooth and continuous audio playback.
- 7) Demonstrated a full analog-to-digital-to-analog signal conversion process within a single integrated system.

A. Future Improvements

Future enhancements may include upgrading the DAC resolution to improve audio quality, adding an audio amplifier for clearer sound output, supporting multiple audio files and formats, and incorporating an LED indicator or display module to provide user feedback during system operation.